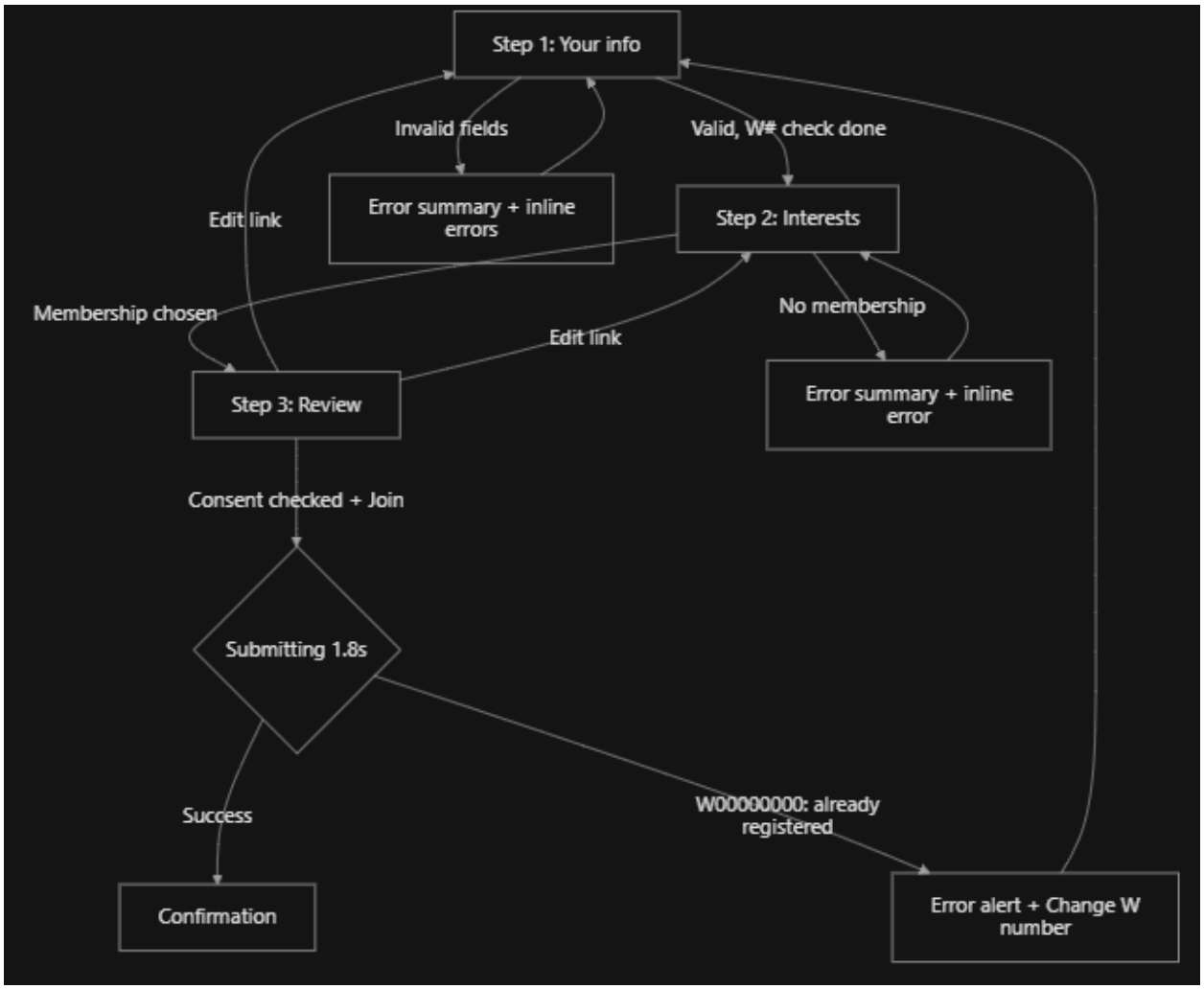
1. Flow Selection and Input Mechanics

1.1 User Flow Selection

A three-step registration wizard with a confirmation screen.



Steps	Screen	Controls
1.	Your Info	Full name, W number, email, class year (select)
2.	Interests	Membership type (radio group), interests (checkbox group), collapsible needs field
3.	Review and Join	Summary with Edit buttons,

		consent checkbox, Join button
4.	Conformation	Success message, "Register another person"

1.2 Input paradigm map

Paradigm	**How the flow supports it**
Touch	Every control is at least 48px tall. Whole option rows are tappable, not just the 24px radio or checkbox. Buttons have 12px gaps. No hover-only content and no gesture-only actions.
Pointer	Hover states on inputs, option rows, and buttons. The pointer target equals the visual target.
Keyboard	Tab and Shift+Tab move through controls in DOM order. Space toggles checkboxes and activates buttons. Arrow keys move within the radio group. Enter activates buttons and submits the form. A skip link jumps to the form.
Screen reader	Each step moves focus to its <h2>, and a polite status region announces "Step 2 of 3: Interests". Errors move focus to a summary with jump links. Groups are announced through <fieldset>/<legend>. The progress list exposes aria-current="step".
Voice control	Visible labels match accessible names (for example "Continue to interests"), so "click Continue to interests" works. Edit buttons have unique aria-labels ("Edit W number").

1.3 Fitts' Law Target Sizing

Element	**Hit area**	**Spacing to neighbor**	**Meets 44px / 48px**

Text inputs, select	full width × 48px min	20px between fields	Yes
Radio/checkbox option rows	full width × 48px min (control itself 24×24, label is the target)	8px between rows	Yes
Primary and secondary buttons	48px min height and width	12px between buttons	Yes
Needs toggle (disclosure)	full width × 48px	8px to the panel	Yes
"Edit" buttons on review	48px min height and width	20px or more between rows	Yes
Skip link	48px min height	n/a	Yes

2. Component State Matrix

Component	**Default**	**Hover**	**Focus**	**Active**	**Disabled**	**Loading**	**Error**
Text input / select	2px #66756E border, white fill	Border darkens to #1B2420	3px #0B57D0 outline, 2px offset	n/a (native)	n/a	n/a	Red #B3261E border, #FCEDEC fill, aria-invalid="true", red inline message below
W# input	As above	As above	As above	n/a	/na		Format message, or "already registered" after submit
Radio / checkbox row	Bordered row, white fill	Gray-green fill, darker border	Outline on the whole row via	Pressed fill #E3EAE6	n/a	n/a	Red left border on the fieldset,

			:has(:focus-visible)				inline message
Primary button	Pine #1F4D3F fill, white text	Darker #163A30	3px blue outline, 2px offset	Darkest #0F2A22, shifts 1px down	n/a	n/a	n/a
Secondary button	White fill, pine border and text	#E6F0EB fill	3px blue outline	#D3E4DB fill	n/a	n/a	n/a
Needs toggle	Secondary style, chevron right, aria-expanded="false"	As secondary	As secondary	As secondary	n/a	n/a	n/a
Edit button	Underlined pine text	Inherits hover fill	3px blue outline	Pressed fill	n/a	n/a	n/a
Join button	Primary style	Primary hover	Primary focus	Primary active	aria-disabled="true": gray fill, "not-allowed" cursor, reason in #submit-note linked by aria-describedby	Spinner + "Joining …", aria-disabled="true", form aria-busy="true"	Re-enabled, error alert with recovery button
Error summary	Hidden	n/a	Receives focus when shown (tabindex="-1")	n/a	n/a	n/a	Red box listing each problem as a jump link

3. Accessibility Architecture

3.1 Semantic structure (all screens)

Screen	Landmarks and grouping
Step 1	<section> labelled by its <h2>. Each field is a <label for> + <input> pair with autocomplete tokens (name, email)
Step 2	Two <fieldset>/<legend> groups (membership radios, interests checkboxes). The needs toggle is a <button> that controls a panel
Step 3	A <dl> summary of the entered data, a consent checkbox inside a <label>, and a submit <button type="submit">
Confirmation	<section> whose <h2> receives focus

3.2 ARIA and live regions

Hook	Where	Purpose
aria-expanded + aria-controls	Needs Toggle	Announces collapsed/expanded and the panel it controls
role="status" / aria-live="polite"	#status	Step changes and submitting messages
aria-live="polite"	Each inline .err and the W# check line	Announces field errors and "W number looks good" without interrupting
role="alert" (assertive)	Error summaries	Immediately announces that submission or step validation failed
aria-invalid	Invalid inputs	Marks the field state for assistive tech
aria-describedby	Inputs, Join button	Links hints, errors, and the disabled reason to their controls
aria-busy	<form> during submit	Signals the form is processing

aria-current="step"	Progress list	Identifies the current step
aria-label	Edit buttons	Makes "Edit" unique per field

3.3 Contrast audit (WCAG 2.1 AA)

Pair	Foreground	Background	Ratio	Requirement	Pass
Body text	#1B2420	#FFFFFF	15.91	4.5	Yes
Body text on page	#1B2420	#EEF2EF	14.08	4.5	Yes
Hint text	#46554E	#FFFFFF	7.87	4.5	Yes
Hint text on page	#46554E	#EEF2EF	6.96	4.5	Yes
Primary button	#FFFFFF	#1F4D3F	9.58	4.5	Yes
Primary hover	#FFFFFF	#163A30	12.49	4.5	Yes
Primary active	#FFFFFF	#0F2A22	15.28	4.5	Yes
Header subtitle, step labels	#DDE9E3	#1F4D3F	7.68	4.5	YEs
Secondary button	#1F4D3F	#FFFFFF	9.58	4.5	Yes
Secondary hover	#1F4D3F	#E6F0EB	8.23	4.5	Yes
Error text	#B3261E	#FFFFFF	6.54	4.5	Yes
Error text on error fill	#B3261E	#FCEDEC	5.75	4.5	Yes
Success text	#17663A	#FFFFFF	7.00	4.5	Yes
Input border (UI	#66756E	#FFFFFF	4.84	3.0	Yes
Focus ring on card (UI)	#0B57D0	#FFFFFF	6.39	3.0	Yes

Focus ring on page (UI)	#0B57D0	#EEF2EF	5.65	3.0	Yes
Current step bar (UI)	#FFFFFF	#1F4D3F	9.58	3.0	Yes
Inactive step bar (UI)	#7FA597	#1F4D3F	3.53	3.0	Yes
Checked option border (UI)	#1F4D3F	#E6F0EB	8.23	3.0	YEs
Disabled button text	#4F5D56	#DCE2DE	5.27	3.0	Yes

4. Latency and Error Handling

Moment	**Delay**	**Feedback**
W number availability check (on blur, valid format)	1.0s	Spinner + "Checking W number…", then "W number looks good." in a polite live region. Continue is blocked with an announcement if the check is still running.
Submit	1.8s	Button shows a spinner and "Joining…", is set aria-disabled, form is aria-busy, and a polite message "Submitting your registration. Please wait." is announced. The submit handler also exits early while submitting is true.
Submit failure (W00000000)	n/a	role="alert" summary explains the problem, button is re-enabled, and "Change W number" returns focus to the W# field with an inline error.

5. Post-Mortem Reflection (Metacognition)

How did explicitly defining every component state (Hover, Focus, Active, Loading, Error) during the specification phase prevent dynamic usability bugs that static wireframes completely fail to address?

A wireframe shows one frame of the Join button, so it doesn't show the edge cases of what happens when the user clicks it twice or presses it before agreeing to the waiver. Writing out the full state row for that button exposed several problems before any bug existed. The disabled state needed a reason, which led to aria-disabled rather than disabled so keyboard and screen reader users can understand the explanation. The static mockup never shows where keyboard focus goes after a step change or a failed validation, so defining the focus state forced me to decide that each step moves focus to its heading and each error moves focus to a summary.

Describe a scenario in your prototype where system latency or asynchronous data processing required specialized micro-interaction feedback (e.g., loading skeletons or aria-live announcements). How does your design keep the user informed and prevent repeated submission errors?

For the submit step, joining takes about 1.8 seconds in the prototype, which is long enough that, without any feedback, a user might think nothing happened and click again.When the user presses Join, the button immediately swaps its label for a spinner and "Joining…", becomes aria-disabled, and the form is marked aria-busy. A live region announces that the submission is in progress, so screen reader users can get the same information sighted users get from the spinner. A submitting flag in the handler ignores any further click or Enter press until the request finishes, so a double submission can't happen even if someone bypasses the visual state. If the server responds with an error, the button is restored and an alert explains the problem and offers a one-click fix.